

Lesson 0

Introduction to APIs

Building backend services to transform teaching materials in real products

Welcome to the AI Lab



Collaborative Space for Teams

Experiment, learn, and build cutting-edge AI solutions together in a dedicated environment.



High-Performance Laptops

25 laptops are readily available to access different services and experiment without heavy restrictions.



AI Services Provided by TCS

Internal gateway enables the use of multiple corporate-provided LLM models in a highly secure sandbox environment.



LiteLLM Gateway

What is the LiteLLM Gateway?

The company's unified platform that acts as a central facade to consume multiple AI providers (OpenAI, Anthropic, Google) under a single standardized interface.

It centralizes credentials, load balancing, budget limits, and token usage metrics.

LiteLLM Swagger Interface

Interactive demonstration: Live web documentation exposing all available endpoints in the AI Lab.

Allows testing prompts, adjusting parameters, and verifying responses directly from the browser before coding.

From the Notebook to a Real Product

Jupyter Notebooks as Exploratory Environment

Ideal for research, data analysis, and interactive cell-by-cell prompt tuning.

Limitations: Synchronous local execution, exposed credentials, and no multi-user capability.

The Backend Server as Useful Product

Persistent web service exposing a clear contract to be consumed by a Frontend or an AI Agent.

Benefits: Hides technical complexity, protects confidential information on the server, and handles traffic for multiple users at the same time.



LiteLLM from TCS shares the same principle: The company wanted to facilitate the access to some resources to other people while holding control of the platform.

Lesson 1 Objectives

1. Web APIs & HTTP Protocol

Understand what an API is, the Client-Server model of the web, the request/response anatomy, and the communication between the parts.

2. FastAPI & Proxy Pattern

Structure web services with Pydantic, generate Swagger documentation, and hide LiteLLM API Keys in the backend.

3. FastAPI Layer Architecture

Prevent the chaos of having all the backend implementation in a single file by structuring it cleanly into layers.

Section 1

Web APIs & HTTP Protocol

Understand what an API is, the Client-Server model of the web, the request/response anatomy, and the communication between the parts.

What is an API?



Application Programming Interface

It defines a **contract** detailing how two software components communicate with each other, specifying which information should be shared between the two parts involved in the communication.

- What happens if the information shared is not the agreed in the contract? It fails.



What is an Interface?

An **interface** is an **abstraction layer**: it provides the access to a tool without needing to care about the internal mechanics of the tool, allowing users to use it without a deep understanding of how it works.

Other Examples of Interfaces:

- **Hardware Interface** (e.g., USB ports, HDMI connectors)
- **User Interface (UI)** (e.g., end-user Applications, Web Pages)



Discussion Topic: Which is the interface that humans use everyday with the same purpose?

API Examples: Local vs. Remote



1. Local APIs

Operate directly within local memory to connect libraries.

For example, calling `pd.read_csv('data.csv')` in Pandas abstracts the file parsing and data transformation that requires converting a comma separated file into a Pandas Dataframe.



2. Remote APIs

Operate remotely over a network using HTTP.

For example, calling a URL to get a list of conversations in a chat application abstracts network connections, database retrievals, and multiple cloud connections that are required to retrieve that information.



Discussion Topic: *Given that it hides complexity, isn't even a simple function that encapsulates logic within a program of code an API?*

Web API: The API on the Network

Communication over HyperText Transfer Protocol

Specialization of the API concept where interaction occurs over a network via standard web protocols (HTTP/HTTPS).

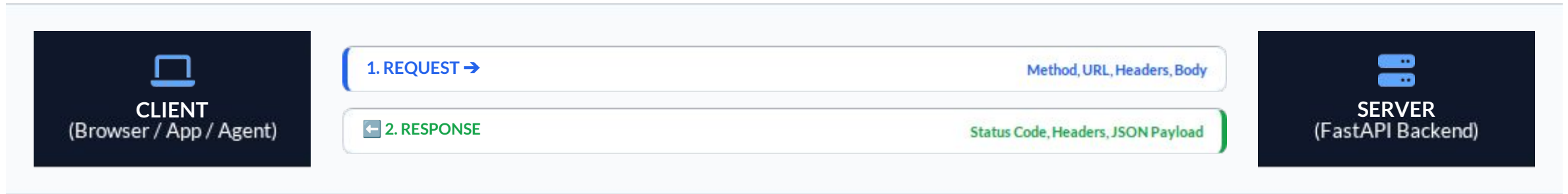
Language Independence & Interoperability

Allows a another service, a client written in a different language, or an AI Agent to connect seamlessly with a backend developed in Python.

Universal JSON Structure

Lightweight, portable, and universally human-readable standard format for structured data exchange between clients and web servers.

The Flow of Information on a WebAPI: Request and Response



Request: Issued by the client. Contains target URL, required action, credentials, and data payload.

Response: Returned by the server. Includes the numeric status code and processed data in JSON.

HTTP Request Example

```
// Headers
POST /v1/chat HTTP/1.1
Host: https://genailab.tcs.in/
Authorization: Bearer tcs_key_abc123
Content-Type: application/json

// Payload (JSON Body)
{
  "prompt": "What is an API?",
  "temp": 0.7
}
```

HTTP Response Example

```
// Headers
HTTP/1.1 200 OK
Content-Type: application/json
Server: FastAPI/0.100.0

// Payload (JSON Body)
{
  "response": "An API is a formal contract..."
}
```

Understanding HTTP Headers

What are HTTP Headers?

HTTP headers act as vital metadata for network transactions. They allow the client and server to pass essential context and configuration alongside an HTTP request or response.

Key Architectural Rule

Headers are completely distinct from the payload body and are handled by network proxies, gateways, and security layers.

Crucial Headers for Development

Content-Type: application/json

Informs the backend server or LLM orchestrator that the payload is formatted as structured JSON metadata, ensuring proper parsing on both ends.

Authorization: [API_KEY]

Securely transmits API credentials. This is mandatory for authorizing your requests when communicating with any other external service that you want to integrate to your own one, like the GenAI Lab's endpoints by TCS or any other third party service that works like a SaaS platform.

Request Types according to HTTP

Method	Primary Purpose	Where is data sent?	Example
GET	Read / Query information without modifying the server.	Query Params in URL (?model=gpt4)	Query available AI models list.
POST	Send data for processing or resource creation.	Request Body (structured JSON)	Send a complex prompt with requested temperature.
PUT / PATCH	Full update (PUT) or partial update (PATCH) of data.	Request Body (JSON) + URL ID	Modify an existing DB entry.
DELETE	Resource deletion on the server.	URL / Query Params	Delete a chat session history.

HTTP Request Headers Example

```
// Headers
POST /v1/chat HTTP/1.1
Host: https://genailab.tcs.in/
Authorization: Bearer tcs_key_abc123
Content-Type: application/json
```

Response Status Codes according to HTTP

Family	Category	Semantic Meaning	Common Examples (RFC Standard)
1xx	Informational	Request received, process ongoing.	100 Continue, 101 Switching Protocols
2xx	Success	Action received and processed successfully.	200 OK, 201 Created, 204 No Content
3xx	Redirection	Client must take additional action.	301 Moved Permanently, 307 Temporary Redirect
4xx	Client Error	Malformed request, missing credentials, or nonexistent resource.	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Server Error	Server failed while processing a valid request.	500 Internal Error, 503 Service Unavailable

HTTP Response Example

```
// Headers
HTTP/1.1 200 OK
Content-Type: application/json
Server: FastAPI/0.100.0
```

Enforcing of the Response Status Codes

Is There Technical Enforcing or Obligation?

There is no compiler or "police" on the web: The HTTP protocol relies on standardized agreements and conventions (RFCs). Technically, nothing prevents you from returning 200 OK with an error body or using GET to delete records.

Why is it MANDATORY to follow rules in practice?

- 1. Monitoring & Telemetry:** APM tools (Datadog, Kibana) track system health by counting 4xx/5xx status codes. Returning with 200 OK hides failures and provides false positives.
- 2. AI Agents & Code Clients:** Libraries or AI SDKs execute verification on numeric codes. Returning 200 causes AI to treat error messages as valid outputs.

Confusion Happens: the 401 vs 403 case

401 Unauthorized: Lack of Authentication (no credentials provided / identity unknown).

403 Forbidden: Lack of Authorization (identity known, but lacking permission for this resource).



Consult MDN Web Docs Official HTTP Status Code Documentation



The Message: JSON Format

What is JSON?

Lightweight, text-based format for structured data exchange, universally readable by humans and machines.

```
// Key-Value Format
data = {
    "role": "assistant",
    "tokens": 150,
    "active": true
}
```

The standard language for client-server and API communication.

In Python terms

Encoding (Serialization)

Converts native Python dicts/lists into a standardized JSON string using **dumps()**.

```
# Python dict to JSON string
json_str = json.dumps(data)
```

Decoding (Deserialization)

Converts a JSON string back into Python dictionaries/lists using **loads()**.

```
# JSON string to Python dict
data = json.loads(json_str)
```

** Note: The 'json' module must be imported.*

Why JSON is Crucial in AI

Ensures predictable schema structures for non-deterministic LLM systems integration.

Tool Calling

Enables LLMs to output structured arguments to invoke web APIs dynamically.

Structured Outputs

Enforces schema-valid configurations to guarantee deterministic formats.

 **Note:** In FastAPI, Pydantic models automatically handle JSON validation, serialization, and deserialization.

Section 2

FastAPI & Proxy Pattern

Structure web services with Pydantic, generate Swagger documentation, and hide LiteLLM API Keys in the backend.

Why Jupyter is Not Viable for Production

While excellent for prototyping, notebooks present critical vulnerabilities and usability issues for end-user products.

1. Security & Credentials

Exposed Endpoints: Execution exposes third parties information to the end user.

Hardcoded Keys: Credentials could be easily extracted.

2. Scaling & Architecture

Single client: Cannot handle multiple requests at the same time.

3. End-User Experience

Out of Order Becomes Impossible: The execution of a notebook is linear.

Poor State Resilience: Jupyter kernels tend to freeze or crash without failovers.

Backend as an Orchestrator & Proxy

Transitioning from vulnerable experimental sandboxes to secure, production-grade API architecture.

Service Orchestration

Coordinates and unifies application layers under a single managed flow.

Auth Services: Manages credentials and permissions.

Databases: Queries SQL, NoSQL, and vector indexes.

AI Gateways: Structures and dispatches requests to AI models services.

Why It Acts as a Proxy

Acts as a protective, governed shield between clients and back-end providers.

Credential Hiding: Prevents exposure of API keys and secrets.

Payload Validation: Blocks malformed requests using Pydantic.

Decoupling: Hides backend complexity behind endpoints like /chat.

Production Advantages

Resolves core bottlenecks and safety risks of interactive Jupyter notebooks.

Concurrent Scaling

Supports thousands of parallel requests async.

State Resilience

Eliminates kernel locks and out-of-order execution.

System Controls

Adds enterprise caching, rate limits, and audit logs.

Client Authentication Methods

Backend services require robust validation to identify and authorize API clients before granting access.

1. API Keys & Secrets

Stateless Access: Simple, unique tokens passed in headers (e.g., X-API-Key) to identify the calling application.

Use Case: Perfect for server-to-server communication and public data endpoints where user context is not needed.

2. JSON Web Tokens (JWT) / OAuth 2.0

Claims-Based: Cryptographically signed payloads issued by Identity Providers (e.g., Auth0, Entra ID) containing scopes and permissions.

Use Case: Standard for modern web/mobile apps to secure APIs on behalf of authenticated, individual users.

Why Authentication Matters for AI Services

Foundation for Production: Essential for securing from unauthorized access, tracking resources consumption, and enforcing rate-limits to prevent runaway API costs.

Python Backend Frameworks



Django

Traditional, comprehensive framework with built-in database tools. Well-suited for standard websites, but heavy and slow for modern, lightweight intelligence services.



Flask

A simple and minimal starter framework. Popular and easy to use, but processes requests one-by-one and lacks built-in verification rules.



FastAPI

Built specifically for running multiple tasks at the same time. Fast, automatically checks incoming data, and is the perfect fit for serving artificial intelligence models.

Hands-on: Hello World

Step 1

Environment Setup

Create the Python's virtual environment and install the required packages for today's assessment.

Step 2

Hello World in FastAPI

Create a Hello World endpoint and test it using the FastAPI-provisioned test client at `/docs`.

Pydantic Validation



What is Pydantic? — A powerful data validation library built on Python type annotations. As [FastAPI's core foundation](#), it enables seamless request parsing, strict runtime enforcement, and automatic interactive documentation.



Automatic 422 Errors — FastAPI automatically rejects requests containing malformed data or missing parameters with a **422 Unprocessable Entity**, eliminating manual check code.



Proxy Validation — Ensures conversations with an LLM conform to the strict expected schema. Without this defined structure, the LiteLLM request will be rejected immediately, provoking downstream client errors.

Hands-on: Chat Application

Step 3

Small Chat API

Explore a very simple example of how a chat application could look like and show different APIs and HTTP concepts in practice, together with an example of how authentication works.

Hands-on: Authentication and Authorization

Step 4

Adding Authentication to the Chat app

An example of access restrictions to the chat application.

Section 3

FastAPI Layer Architecture

Preventing the chaos of having all the backend implementation in a single file

Why Layered Architecture Matters

Decoupling backend responsibilities prevents chaotic codebase growth and accelerates scaling



Separation of Concerns

Each layer has a single, well-defined responsibility. Isolating different parts of the application ensures that changes to user interfaces, business logic, or data storage remain completely decoupled and easier to manage.



Future-Proof Flexibility

Swapping out components becomes effortless. Migrating from SQL to NoSQL, or moving from FastAPI to another framework, only impacts a single, isolated layer.



Simplified Testing

By isolating business calculations from network infrastructure, you can easily unit test critical operations using simple mocks instead of complex database setups.

The Three Backend Layers

How structural compartmentalization keeps code clean, maintainable, and scalable



Routers

Manages network endpoints and HTTP routing. Receives incoming requests, handles path operations, and forwards payloads down.



Services

The core business logic. Processes business rules, performs calculations, manages state, and orchestrates API requests.



Schemas

Defines data models and validation structures using Pydantic. Guarantees data integrity by serializing and parsing payloads.



Alternative Concepts: Other frameworks decouple the logic in different ways and you could find [Controllers](#) taking Routers logic, and [Models](#) acting like FastAPI's Schemas (for example).



PROJECT DIRECTORY

```
my_fastapi_project/  
├── app/  
│   ├── main.py  
│   └── routers/  
│       └── router_1.py  
├── services/  
│   └── service_1.py  
├── schemas/  
│   └── schema_1.py  
└── requirements.txt
```

Hands-on: Layered Architecture



Task 1 - Create a Folder for Every layer and a main file for that layer: Every layer has its own folder and its main file that provides access to all the resources in that layer.



Task 2 - Move the corresponding code for each layer: Copy the code from the main.py file to the corresponding file in the corresponding layer.



Task 3 - Connect everything: Connect services to schemas, and router to services. Finally, connect routers to the main.py that has the FastAPI client implementation



Task 4 - Test the endpoint from Swagger: If no magic, let's debug!

Questions or Feedback?